
THE DESIGN OF “STRATEGA”: A GENERAL STRATEGY GAMES FRAMEWORK

Diego Perez-Liebana
Game AI Research Group
Queen Mary University London
diego.perez@qmul.ac.uk

Alexander Dockhorn
Game AI Research Group
Queen Mary University London
a.dockhorn@qmul.ac.uk

Jorge Hurtado-Grueso
Game AI Research Group
Queen Mary University London
j.hurtado@qmul.ac.uk

Dominik Jeurissen
Game AI Research Group
Queen Mary University London
djeurissen.paper@web.de

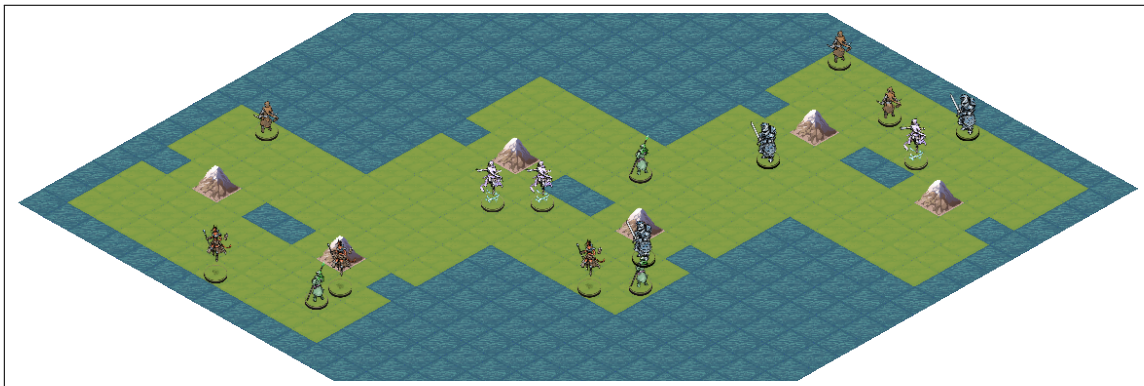
Version 2.0 — April 30, 2026

ABSTRACT

Stratega, a general strategy games framework, has been designed to foster research on computational intelligence for strategy games. In contrast to other strategy game frameworks, Stratega allows to create a wide variety of turn-based and real-time strategy games using a common API for agent development. Flexibility is achieved by utilising YAML-files to configure tiles, units, actions, and levels. Therefore, the user can design and run a variety of games to test developed agents without specifically adjusting it to the game being generated. The framework has been built with a focus of statistical forward planning (SFP) agents. For this purpose, agents can access and modify game-states and use the forward model to simulate the outcome of their actions. While SFP agents have shown great flexibility in general game-playing, their performance is limited in case of complex state and action-spaces. Finally, we hope that the development of this framework and its respective agents helps to better understand the complex decision-making process in strategy games. Stratega can be downloaded at:

<https://github.com/GAIGResearch/Stratega>

Keywords Stratega · General Game AI · Strategy Games · Statistical Forward Planning



Contents

1	How to use this Document?	2
2	The Stratega Framework	2
3	Design and Overview of the Framework	3
4	Game-State	4
5	Forward Models and Actions	4
6	Agents and Algorithms	4
7	Configuration	5
8	Future Directions	7
	References	8

1 How to use this Document?

This paper serves the purpose of a living documentation for Stratega - a general strategy games framework for studying computational intelligence in turn-based and real-time strategy games. We aim to continuously adjust the framework to the needs of our users. This paper will be updated accordingly and describe the inner workings of the Stratega framework as well as the underlying design philosophy. When citing this paper please make sure to specify its version since the paper will be updated over time!

2 The Stratega Framework

Due to their high complexity, strategy games have become an interesting genre for AI research [1]. Fundamental characteristics can include, but are not limited to: complex state and action spaces, partial observability, spatial and temporal reasoning, competition and collaboration in multiple-player settings, player modelling, among others.

Existing strategy game frameworks, e.g. Starcraft [2, 3, 4] and μ RTS [5], focus on studying a single game. This not just allows to incorporate domain knowledge during AI development but also indefinitely training the agent to become proficient in this game. It has been shown that this approach can result in strong AI agents capable of defeating even the best human players, e.g. the AlphaStar agent for Starcraft II [6]

Due to these recent successes in development of single strategy game AI, we believe the time is right to propose a general strategy games framework. Using a similar approach as general game-playing frameworks such as GVGAI [7] and GGP [8], the Stratega framework allows to create a variety of strategy games using a common API for AI development. Stratega allows the creation of n-player turn-based and real-time strategy games with full access to the game’s forward model. While other general game frameworks may be able to generate similar games, the strong focus of Stratega allows to incorporate higher level concepts which ease the development process. In this paper, we are outlining the design of the Stratega framework and discuss its future prospects.

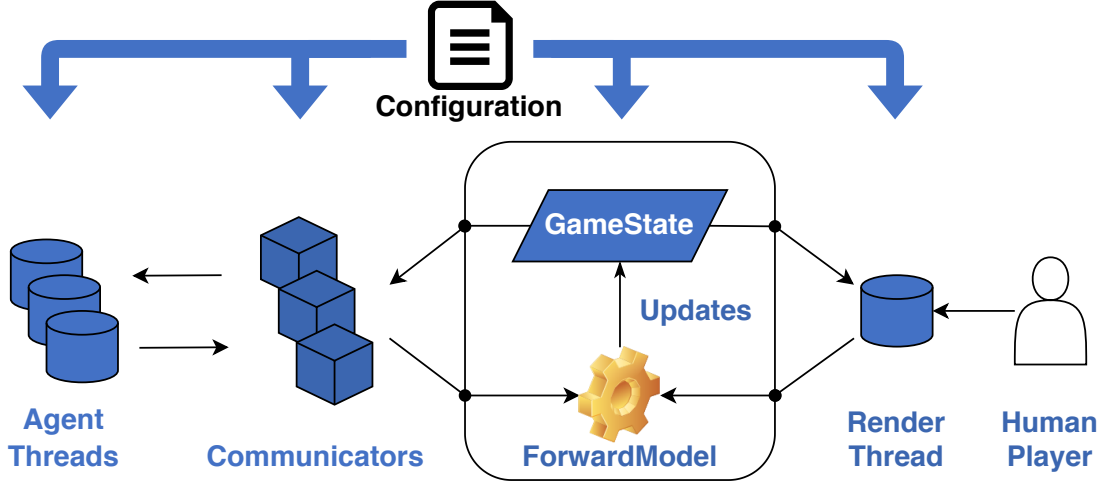


Figure 1: Schematic overview of the framework’s architecture.

3 Design and Overview of the Framework

The Stratega framework mainly consists of 4 parts, i.e. the game-runner, rendering, agents, and configuration.

The core of the framework is the game-runner, whose sole purpose is to run games and provide information about the game. The framework provides two game-runners, one for turn-based and one for RTS-games. The agents are responsible for controlling the actions of a player in the game state. To do this, each agent receives a game communicator to send actions or poll information about the current game state. Additionally, each agent has access to a copy of the forward model of the game-runner. Regardless of the type of game, every agent-thread runs until the game is over. Agents can query information or send actions at any point in time, even during an opponent’s turn. This enables agents to think outside their own turns. Additionally, the agent can observe how the game evolves to think ahead. Note that the turn-based game-runner ignores actions from players when it’s not their turn. Meanwhile, the RTS-runner will collect the actions from all players and execute them in order of receipt after a set amount of time.

The runner keeps track of which agent controls which player in the game state. If no agent is available for a player, the runner will send a signal and wait until an external source provides the missing actions. The render-thread subscribes to events from the game-runner to update the window. If the runner signals missing agents, the GUI will switch to interactive mode to let human-players play the game. That also has the added benefit that human-players have no time limit. This approach has similarities to the model–view–viewmodel (MVVM) pattern, in the sense that the game-runner represents the view-model. The renderer (view) is decoupled from the logic and can be easily disabled to switch between headless-execution and GUI-execution.

A feature of the framework is the ability to write algorithms for various degrees of game-abstractions. While game-states represent nodes in a game-tree, the forward model represents the edges to get from one node to another. That means the forward model provides actions, as well as the game-logic. Developers can provide a new forward model for the default game state or provide their new representations. Algorithms implemented in the framework can work with any kind of forward model.

Lastly, the framework provides the ability to configure games via YAML-files, which are explained in more detail in upcoming sections. It’s noteworthy that configuration is separated from the rest of the framework, meaning the configuration initialises and sets up the game, but the framework does not need to be re-compiled every time a change in the configuration is made.

4 Game-State

The Game-state object serves as a data container without any logic. It offers access to the current board, its tiles, a list of players, and their units as well as definitions of action and entity types. Due to the strict separation of data and game logic, players receiving a game-state object can freely copy or modify it.

The board is represented as an $n \times m$ -grid of tiles. A tile can be occupied by a unit and will therefore be blocked for other units. Tile types and board layouts are defined via the configuration file (see Section 7). The player's observation of the board is limited by the line of sight of the player's entities. Each unit has a perception radius which removes the fog of war and shows the real content of a tile. Tiles outside this perception radius will be replaced with a default tile. Alternatively, games can be configured to track the agents knowledge of a tiles last known state, which will be returned in case the none of the agent's units is close enough to observe a tile's current state.

An entity is a player controllable character or building which is able to perform one or multiple actions per turn. Each player controls all its entities at once, meaning that during a single "turn" the agent may return multiple actions. In terms of real-time strategy games, we consider a turn to be the time-span until an agent returns a complex action for all its entities. In contrast to the turn-based game-mode, agents are acting simultaneously. This can cause actions to be not applicable anymore. Such actions will be ignored by the forward model.

5 Forward Models and Actions

The forward model is the driving component of this framework. Being provided with a game-state it will generate the set of available actions. Returned action objects store the type of action and a list of sources and targets the action will be applied to. Those can differ based on the action definition. The set of typical sources and targets include but are not limited to: the player's, entities and map elements. Nevertheless, they do not include the action's logic which remains part of the forward model.

When a player provides the forward model with an action and a game-state, it first checks if the action is applicable. This needs to be done, since the player may have modified either the action or the game-state in between generating the set of actions and simulating its outcome. These security checks can be deactivated to increase the framework's speed. In case the action can be executed, the game-state object will be advanced accordingly.

Implemented actions include a movement and an attack action as well as special abilities such as healing units among others. The way in which actions are to be executed is defined in the forward model, which is set up using the configuration files. This allows to quickly change the action's parameters, and therefore, create variants of the same game. This approach has been chosen to easily modify the game's balance without updating the code.

Additional flexibility is achieved by the use of triggers and effects. Given an event, e.g. a unit entering a cell or a player ending a turn, the forward model can execute subscribed methods. This can be used to implement game-mechanics such as a unit losing health at the end of each turn. Alternatively, it allows to log information every time an event occurs.

Both, the forward model and the game state, can be extended by the user to create action and game-state abstractions. This can be useful for reducing the game's complexity and ease the training and decision-making process of AI agents. Similarly, search-based agents may achieve higher performance due to the reduced search space of such an abstraction.

6 Agents and Algorithms

The framework provides access to a small set of standard agents which will be extended over time. For pre-defined game-modes we provide a rule-based agent which can be used as a baseline during evaluation of developed agents. Those use knowledge of the specified game-mode and therefore cannot be universally applied. However, studying their inner workings can be a great introduction into developing agents for our framework. Additionally, we provide several scoring functions which can be used in conjunction with search-based agents.

We provide basic implementations of classic search algorithms, such as One-step look-ahead, Monte Carlo method, Monte Carlo Tree Search [9] (MCTS) as well as Rolling Horizon Evolutionary Algorithm [10] (RHEA). While MCTS and RHEA have shown to perform well in general game-playing tasks [7], they seem to struggle with the higher complexity of strategy games. Initial experiments have shown that the performance of both search-based agents is still limited in comparison to the sample rule-based agents [11], which motivates further research on extending their performance.

To improve the performance of search-based agents, the framework provides access to several portfolio methods [12]. The reduction of the action space’s complexity has shown to improve agent performance in strategy games [13, 14]. First experiments with turn-based games of the Stratega framework show that search-based agents using a state abstraction performed better than agent’s using the whole action space [?].

The recent introduction of game-state abstraction can improve the agent performance in a similar way. The framework provides baseline implementations for state abstractions including a configurable vector-based state representation. The definition of an abstract heuristics works in the same way and provides agents with a general way for optimising weighted game-state evaluation functions for unknown game-states.

7 Configuration

In this section, we will have a closer look on the YAML configuration file. It allows users to adjust the balance of an existing game or define new game modes by simply modifying the game-components through keywords. An exemplary configuration file is shown in Figure 2.

First, we will take a look at the definition of tiles and board. Similar to the level description in the video game definition language [15], we first define tiles and their encoding in the tilemap. Each tile definition consists of a symbol, a sprite, if it is walkable and if it is blocking the view. Further tile attributes will be added during in upcoming versions of the framework.

Later on, the board can be manually described in terms of a tile-map, an n -by- m grid of tiles. Alternatively, it will be possible to procedurally generate levels. Therefore, the user needs to provide a method which will receive the tile configuration and a set of parameters to generate a level. This allows the framework to be used for research on level generation.

Next, entities are defined in terms of their type and their properties. In contrast to previous versions of the framework, the unit definition can use an arbitrary set of parameters. Parameters will be used by actions as we will see in the next paragraph. Typical parameters include health, movement, and damage. In case of partial observable game, each entity requires to set the line of sight range. Here, each entity can observe tiles with a Euclidean distance equal or less than the given threshold. A player’s view of the board will be limited by the union of all tiles visible by their controlled entities. Finally, each entity receives a list of actions.

The configuration excerpt in Figure 2 shows the definition of a healer and a king. Note, that both implement a different set of parameters and actions. In comparison, the healer is a much weaker unit which is unable to attack. Instead, it can use the action heal which replenishes the health of a damaged unit. The action definitions are separated into another section to make the same action reusable across entity types.

The bottom frame of Figure 2 shows the definition of the special actions Attack and Heal. An actions type shows if it will be applied to players, map tiles, or entities. A cooldown can be used in case the action should not be available every turn. The keywords *Conditions* and *Effects* show how parameters of the executing entity can be used to implement the desired effect. Some baseline effects such as *Attack* and *ModifyResources* simplify the implementation of typical game-play mechanics.

The next part of our configuration consists of the forward model’s properties. Here, we first defined a set of win or lose conditions using the same condition system than actions. With the *Trigger* and *Effects* keywords users can introduce more general game mechanics. An effect’s trigger describes when the effect will be executed. Conditions can further limit the effects to be only applied to conforming units or tiles. Here, the number of movement points are replenished every turn.

GameConfig: - Type: TBS - ActionTimer: 10s - RoundLimit: 100 Agents: - RandomAgent - CombatAgent Tiles: Plain: - Sprite: plain.png - Symbol: . - IsWalkable: true - DefaultTile: true Mountain: - Sprite: rock.png - Symbol: M - IsWalkable: false Board: GenerationType: Manual Layout: - <pre> M M M M M M M M M M M . h0 M M M M h0 k0 M M M . . . M M . . . M M M h1 h1 M M M M h1 . M M M M M M M M M M M </pre>	Entities: Healer: - Sprite: healer.png - Symbol: h - LineOfSightRange: 400 - Parameters: - Health: 40 - HealRange: 2 - HealAmount: 10 - MovementPoints: 5 - Actions: [Heal, Move] King: - Sprite: king.png - Symbol: k - LineOfSightRange: 600 - Parameters: - Health: 400 - AttackRange: 2 - AttackDamage: 100 - MovementPoints: 1 - Actions: [Attack, Move] ForwardModel: LoseConditions: - NoHasKing: - "NoHasEntity(Source, King)" Trigger: - OnTick: - Effects: - "SetToMaximum(- Source.MovementPoints)"
--	---

Actions: Attack: - Type: EntityAction - Cooldown: 1 - Targets: - Target: - Type: Entity - ValidTargets: All - Conditions: - "InRange(Source, Target, Source.AttackRange)" - Effects: - "Attack(Target.Health, Source.AttackDamage)" Heal: - Type: EntityAction - Cooldown: 1 - Targets: - Target: - Type: Entity - ValidTargets: All - Conditions: - "InRange(Source, Target, Source.HealRange)" - Effects: - "ModifyResource(Target.Health, Source.HealAmount)"

Figure 2: An simplified configuration of the game-mode Kill the King.

<pre> Actions: Research: Type: PlayerAction Cooldown: 0 Targets: Target: Type: Technology ValidTargets: All Conditions: - "CanResearch(Source, Target)" - "CanAfford(Source, Target)" TriggerComplete: - "HasElapsedTick(Target.Time)" OnStart: - "PayCost(Source, Target)" OnComplete: - "Research(Source, Target)" </pre>	<pre> TechnologyTrees: SingleTree: Mining: Description: Base technology Cost: Prod: 100 Time: 200 Discipline: Description: Build barracks. Requirements: [Mining] Cost: Prod: 150 Time: 200 Archery: Description: Spawn archers. Requirements: [Discipline] Cost: Gold: 2000 Time: 500 </pre>
---	---

Figure 3: Example of a simple technology tree and a research action.

While not shown in this simple example, the configuration file allows to setup entity groups. This can be used to define filters such as buildings, units, attackable targets, etc. The *ValidTargets* field of an action can specify such an entity group to filter the set of applicable entities.

Due to popular request, we added the definition of research actions, technologies and technology trees. Those allow the modification of an entities parameters once they have been researched by a player and can be inserted as additional requirements for action definitions. Figure 3 shows the definition of a research action and a simple chain of technologies. Each technology can define other technologies as requirements. For each player, the set of researched technologies will be tracked and can be used as additional filters of actions available to the player.

In upcoming versions of our framework, we will extend the set of available keywords to ease the development of diverse game-modes.

8 Future Directions

Our aim is to provide a framework for search-based and reinforcement learning agents in complex environments. Additionally, we believe that the way in which game variants can be configured make the framework suitable for research in game and level generation, and automatic game tuning/balancing.

In its current state the framework fully supports tactical turn-based and real-time strategy games and offers an extensive API for developing AI agents. The framework's inherent graphical user interface and logging capabilities support the user to develop agents and analyse their game-playing capabilities. Further logging capabilities will be included in future updates to facilitate research into explainability of AI agents and their decision-making process in complex environments.

In the future, we plan to extend the framework by many facets of modern turn-based and real-time strategy games. Hence, our road-map includes the incorporation of tactical role-playing elements such as inventory management, diplomacy options between players, and temporary effects. While this list will be updated over time, these components would allow to model a large variety of strategy games.

Acknowledgements

This work is supported by UK EPSRC research grant EP/T008962/1.

References

- [1] Michael Buro. Real-time Strategy Games: A new AI Research Challenge. In *IJCAI*, volume 2003, pages 1534–1535, 2003.
- [2] BWAPI Development Team. The Brood War API (BWAPI) 4.2.0. <https://github.com/bwapi/bwapi>, 2020.
- [3] Oriol Vinyals, Timo Ewalds, Sergey Bartunov, Petko Georgiev, Alexander Sasha Vezhnevets, Michelle Yeo, Alireza Makhzani, Heinrich Küttler, John Agapiou, Julian Schrittwieser, et al. Starcraft II: A new Challenge for Reinforcement Learning. *arXiv preprint arXiv:1708.04782*, 2017.
- [4] Santiago Ontanón, Gabriel Synnaeve, Alberto Uriarte, Florian Richoux, David Churchill, and Mike Preuss. A survey of real-time strategy game AI research and competition in StarCraft. *IEEE Transactions on Computational Intelligence and AI in games*, 5(4):293–311, 2013.
- [5] Santiago Ontañón, Nicolas A. Barriga, Cleyton R. Silva, Rubens O. Moraes, and Levi H. S. Lelis. The first microRTS artificial intelligence competition. *AI Magazine*, 39(1):75–83, March 2018.
- [6] Oriol Vinyals, Igor Babuschkin, Junyoung Chung, Michael Mathieu, Max Jaderberg, Wojtek Czarnecki, Andrew Dudzik, Aja Huang, Petko Georgiev, Richard Powell, Timo Ewalds, Dan Horgan, Manuel Kroiss, Ivo Danihelka, John Agapiou, Junhyuk Oh, Valentin Dalibard, David Choi, Laurent Sifre, Yury Sulsky, Sasha Vezhnevets, James Molloy, Trevor Cai, David Budden, Tom Paine, Caglar Gulcehre, Ziyu Wang, Tobias Pfaff, Toby Pohlen, Dani Yogatama, Julia Cohen, Katrina McKinney, Oliver Smith, Tom Schaul, Timothy Lillicrap, Chris Apps, Koray Kavukcuoglu, Demis Hassabis, and David Silver. AlphaStar: Mastering the Real-Time Strategy Game StarCraft II. <https://deepmind.com/blog/alphastar-mastering-real-time-strategy-game-starcraft-ii/>, 2019.
- [7] Diego Perez-Liebana, Jialin Liu, Ahmed Khalifa, Raluca D Gaina, Julian Togelius, and Simon M Lucas. General Video Game AI: A Multitrack Framework for Evaluating Agents, Games, and Content Generation Algorithms. *IEEE Transactions on Games*, 11(3):195–214, 2019.
- [8] Michael Genesereth, Nathaniel Love, and Barney Pell. General Game Playing: Overview of the AAAI Competition. *AI magazine*, 26(2):62–62, 2005.
- [9] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A Survey of Monte Carlo Tree Search Methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, mar 2012.
- [10] Diego Perez, Spyridon Samothrakis, et al. Rolling horizon evolution versus tree search for navigation in single-player real-time games. In *Proceedings of GECCO*, pages 351–358, 2013.
- [11] Alexander Dockhorn, Jorge Hurtado-Grueso, Dominik Jeurissen, and Diego Perez-Liebana. Stratega - a general strategy games framework. In *Proceedings of AIIDE*, 2020.
- [12] Levi H. S. Lelis. Planning algorithms for zero-sum games with exponential action spaces: A unifying perspective. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, July 2020.
- [13] Niels Justesen, Tobias Mahlmann, and Julian Togelius. Online Evolution for Multi-action Adversarial Games. In *European Conference on the Applications of Evolutionary Computation*, pages 590–603. Springer, 2016.
- [14] Niels Justesen, Tobias Mahlmann, et al. Playing multiaction adversarial games: Online evolutionary planning versus tree search. *IEEE Transactions on Games*, 10(3):281–291, 2017.
- [15] Tom Schaul. An extensible description language for video games. *IEEE Transactions on Computational Intelligence and AI in Games*, 6(4):325–331, 2014.